Project Report on

'SmartFlow: Real-Time Traffic Congestion Prediction using Apache Spark and Machine Learning'

Submitted in Big Data Analytics and Business Intelligence Course, CSET 371

Bachelor of Technology

Of

Bennett University, Greater Noida

By

Student Name 1	Enrolment No. 1
Student Name 2	Enrolment No. 2
Student Name 3	Enrolment No. 3
Student Name 4	Enrolment No. 4

BENNETT UNIVERSITY (THE TIMES GROUP), GREATER NOIDA

Under the guidance of

Faculty Name

Designation

School of Computer Science Engineering and Technology (SCSET)

SAMPLE REPORT ON A DIFFERENT TOPIC – created only to show the structure and style of the CSET-371 report format. Do not submit, copy or paraphrase any part of it.

DECLARATION

We hereby declare that the project report entitled “SmartFlow: Real-Time Traffic Congestion Prediction using Apache Spark and Machine Learning”, has not been copied or plagiarized from any other source, and all information, data, and materials used in this report have been properly acknowledged and cited in the text. The work has not been submitted previously for any other course project.

PLAGIARISM REPORT CERTIFICATE

Further, the plagiarism similarity index of the report from Turnitin is _____ %, AI plag is _____% which is within the permissible limit of Turnitin and AI must be 0%. All sources of information used in the report have been properly acknowledged and cited.

Student Name: _____ Enrollment No.: _____

Signature of Team members Candidate

Teamwork and Collaboration

> **How this section works:** list every member and what they actually did. Be specific ("built the Kafka producer and streaming job") rather than vague ("helped in coding"). Examiners use this table to ask each member viva questions.

Team member name	Enrollment No	Individual Contribution in Project
Member 1	—	Problem definition, literature review (Sections 1.1–1.6), data collection from PeMS
Member 2	—	System architecture and Spark pre-processing design (Sections 2.1–2.3, 2.8)
Member 3	—	Algorithm selection and evaluation plan (Sections 2.4–2.6, 2.11)
Member 4	—	Tools, experimental setup, diagrams and report formatting (Sections 2.7, 2.9, 2.10)

TABLE OF CONTENTS

Chapter 1: Problem Domain Identification and Analysis 6
1.1 Background of the Study 6
1.2 Introduction to the Problem Domain 6
1.3 Literature Review 7
1.4 Research Gap and Literature Gaps 8
1.5 Review of Existing Solutions 9
1.6 Limitations of Existing Systems 10
1.7 Problem Statement 10
1.8 Proposed Solution Overview 10
1.9 Objectives of the Project 11
1.10 Scope of the Project 11
1.11 Industrial Applications and Use Cases 12
Chapter 2: Project Design and Methodology 13
2.1 System Overview 13
2.2 Proposed System Architecture 13
2.3 Architecture Components and Their Functions 14
2.4 Proposed Methodology 14
2.5 Methodology Workflow Use-case/DFD Flow Diagram 15
2.6 Algorithms and Techniques Used 16
2.7 Dataset Description 16
2.8 Data Collection and Data Preprocessing 17
2.9 Tools and Technologies Used 18
2.10 Experimental Setup and Configuration 18
2.11 Evaluation Metrics 18
References 20

LIST OF FIGURES

Figure 2.1 Proposed system architecture of SmartFlow 13
Figure 2.2 Methodology workflow 15
Figure 2.3 Use-case diagram 15

LIST OF TABLES

Table 1.1 Summary of reviewed literature 8
Table 1.2 Comparison of existing traffic solutions 9
Table 2.1 Architecture components and their functions 14
Table 2.2 Algorithms used and reason for selection 16

Table 2.3 Dataset description 17
Table 2.4 Tools and technologies 18
Table 2.5 Experimental configuration 18
Table 2.6 Evaluation metrics 19

Chapter 1: Problem Domain Identification and Analysis

1.1 Background of the Study

> **How this section works:** give the big picture: why this area matters today, with 1–2 cited facts. Move from general (cities, traffic) to specific (data + prediction). About 1 page. Do not describe your solution yet.

Road transport is the backbone of daily life in most cities. People travel to work, goods move to markets and emergency vehicles reach hospitals on the same roads. As cities grow, the number of vehicles grows faster than the road network can be widened, and the result is congestion: long queues at signals, slow-moving freeways and unpredictable travel times. Congestion wastes fuel, increases vehicle emissions and adds stress to every trip.

At the same time, roads have become a large source of data. Inductive loop detectors buried under freeway lanes, traffic cameras, GPS devices in phones and taxis, and connected vehicles report speed and vehicle counts every few seconds. A single public system such as the Caltrans Performance Measurement System (PeMS) collects readings from thousands of detectors on California freeways and stores them at 5-minute resolution [13]. Over months and years this becomes a very large and continuously growing dataset.

Traditionally, traffic forecasting relied on statistical time-series models such as seasonal ARIMA [5]. These work well for a single road with stable patterns but struggle when thousands of sensors must be modelled together and when patterns change because of weather, accidents or holidays. Research over the last decade has shown that machine learning and deep learning models trained on large historical data can forecast traffic more accurately [6], [7], [8].

Processing this volume of data needs distributed computing. Frameworks such as Apache Hadoop introduced the MapReduce model for batch processing [4], and Apache Spark later made it faster by keeping data in memory and adding libraries for SQL, machine learning and streaming in one engine [1]. This combination of large sensor data, machine learning and distributed processing is the background for the present study.

1.2 Introduction to the Problem Domain

> **How this section works:** define the domain and its key terms so that a reader from outside the field can follow the rest of the report. Short definitions + one example. About 1 page.

The problem domain of this project is **short-term traffic prediction**, which means estimating the traffic condition on a road segment 15, 30 or 60 minutes into the future using current and past observations. Three basic variables describe traffic on a freeway:

- **Flow** – the number of vehicles passing a detector in a time interval (vehicles per 5 minutes).
- **Occupancy** – the percentage of time the detector is covered by a vehicle; higher occupancy means denser traffic.
- **Speed** – the average speed of vehicles in the interval (miles or km per hour).

A road is usually called **congested** when the average speed falls well below the free-flow speed of that segment. In this project a segment is labelled Free-flow, Moderate or Heavy using speed thresholds, which turns part of the problem into a classification task in addition to the regression task of predicting speed.

Traffic data has three properties that make it a Big Data problem. It is large in **volume** (millions of readings per month), it arrives with high **velocity** (every 5 minutes from every sensor) and it has **variety** (sensor readings, weather, holidays and events). A useful solution must therefore process historical data in batches for training and also process live data as a stream for real-time prediction.

1.3 Literature Review

> **How this section works:** summarise 6–10 real papers related to your problem. For each: what they did, data used, result, and one limitation. End with a summary table. Always cite with [n] and list the paper in References. 2–3 pages.

This section reviews research on traffic prediction and on Big Data platforms, in roughly chronological order.

1.3.1 Statistical approaches

Williams and Hoel [5] modelled freeway traffic flow as a seasonal ARIMA process and showed that the weekly pattern of traffic can be captured with a seasonal difference. Their model gave stable forecasts for individual locations. However, ARIMA assumes that the series is linear and stationary after differencing, and a separate model has to be fitted for every sensor, which does not scale to a whole network.

1.3.2 Machine learning and deep learning approaches

Lv et al. [6] used a stacked autoencoder trained on PeMS data and reported better accuracy than shallow models, arguing that deep features capture non-linear spatial and temporal relations. Ma et al. [7] applied a Long Short-Term Memory (LSTM) network to speed data from microwave sensors in Beijing and found that LSTM handles long-range time dependencies better than traditional recurrent networks and statistical models.

Later work added the road network structure. Li et al. [8] proposed the Diffusion Convolutional Recurrent Neural Network (DCRNN), which treats sensors as nodes of a graph and models traffic as a diffusion process. They released two benchmark datasets, METR-LA (207 sensors in Los Angeles) and PEMS-BAY (325 sensors in the San Francisco Bay Area). Yu et al. [9] proposed Spatio-Temporal Graph Convolutional Networks (STGCN), which use convolutions instead of recurrent units and train faster. Both works report strong accuracy but need GPUs and are trained on a single machine.

Tree-based ensembles are also widely used for tabular traffic features. Random Forests [10] and gradient boosting [11] handle non-linear relations and mixed feature types (time of day, weather, holidays) without heavy tuning, and they are available in distributed form in Spark MLlib [2].

1.3.3 Big Data platforms

Dean and Ghemawat [4] introduced MapReduce for processing large datasets on clusters of commodity machines. Zaharia et al. [1] presented Apache Spark as a unified engine whose in-memory model is much faster for iterative workloads such as machine learning. Meng et al. [2] described MLlib, Spark's distributed machine-learning library, and Armbrust et al. [3] introduced Structured Streaming, which lets the same DataFrame code run on live data. For ingesting high-rate event streams, Apache Kafka [12] is the most common message broker.

Table 1.1 Summary of reviewed literature

Ref.	Method	Data	Strength	Limitation
[5]	Seasonal ARIMA	Freeway flow	Simple, interpretable	One model per sensor; linear
[6]	Stacked autoencoder	PeMS	Learns non-linear features	Single machine, no streaming
[7]	LSTM	Beijing microwave sensors	Captures long dependencies	Slow to train; GPU needed
[8]	DCRNN (graph + RNN)	METR-LA, PEMS-BAY	Uses road network	Complex; not real-time
[9]	STGCN	PeMS	Faster than RNNs	Research code, no deployment
[1]–[3]	Spark, MLlib, Streaming	General	Scales to clusters	Not traffic-specific

1.4 Research Gap and Literature Gaps

> **How this section works:** say clearly what the papers in 1.3 did NOT do. Each gap should point to something your project will do. Refer back to the papers by number. About 1 page.

From the review, the following gaps are identified:

1. **Scalability of training.** The most accurate models [6]–[9] are trained on one machine. As the number of sensors and the length of history grow, single-machine training becomes slow and memory-bound.
2. **Batch-only evaluation.** Most studies report accuracy on a fixed test file. Very few show a working pipeline that scores new readings as they arrive and updates a dashboard.
3. **Limited external context.** Weather and holidays strongly affect traffic, yet many benchmark studies use sensor readings only.
4. **Accuracy-only focus.** Papers compare RMSE or MAE but rarely discuss how a traffic officer would actually use the prediction (alerts, dashboards, congestion levels).

This project addresses these gaps by building an end-to-end, distributed pipeline on Apache Spark that combines sensor, weather and calendar data, compares several MLlib models and serves predictions through streaming and a business-intelligence dashboard.

1.5 Review of Existing Solutions

> **How this section works:** describe real products or systems already used for this problem (apps, government systems, company tools). Stay factual and general; don't invent numbers. A comparison table works well. About 1 page.

Several kinds of solutions already exist for traffic information:

- **Navigation apps (e.g. Google Maps, Waze).** They estimate live traffic from the location data of phone users and suggest faster routes. The models and data are proprietary and are designed for individual drivers rather than for city-level planning.
- **Government sensor systems (e.g. Caltrans PeMS).** PeMS collects and archives detector data and provides reports on delay and performance [13]. It is excellent for historical analysis but is not mainly a prediction tool.
- **Adaptive signal-control systems.** Some cities adjust signal timings using detector data at junctions. These react to current conditions at a junction but do not forecast conditions on the wider network.

Table 1.2 Comparison of existing traffic solutions

Solution	Data source	Prediction?	Open / reusable?	Main user
Navigation apps	Phone GPS (crowd-sourced)	Yes (short term)	No – proprietary	Drivers

Solution	Data source	Prediction?	Open / reusable?	Main user
PeMS	Loop detectors	Limited	Data is public	Transport agencies
Adaptive signals	Junction detectors	Reactive only	Vendor specific	Signal operators
SmartFlow (proposed)	Detectors + weather + calendar	Yes, 15–60 min	Open-source stack	Traffic officers, analysts

1.6 Limitations of Existing Systems

> **How this section works:** turn the review in 1.5 into a short list of concrete weaknesses. Keep each point to 2–3 sentences. About half a page.

- **Closed data and models:** commercial apps do not share their models, so agencies cannot audit or adapt them.
- **Reactive rather than predictive:** many operational systems describe the current state but do not warn about congestion that is about to form.
- **Separate tools for batch and live data:** historical analysis and real-time monitoring are often done in different systems, which duplicates effort and causes inconsistent results.
- **Poor scalability of research code:** academic models are usually run as single-machine scripts and are hard to extend to thousands of sensors and years of history.

1.7 Problem Statement

> **How this section works:** one clear paragraph (or a boxed statement) that says what problem you will solve, for whom, with what data, and under what constraints. Examiners often read this section first.

Urban freeway networks generate millions of sensor readings every month, yet transport authorities lack an open, scalable system that can learn from this historical data and predict congestion 15–60 minutes ahead in near real time. The problem addressed in this project is to **design and implement a distributed Big Data pipeline on Apache Spark that ingests freeway sensor, weather and calendar data, trains and compares multiple machine-learning models for short-term speed and congestion-level prediction, and delivers the predictions through a streaming service and an interactive dashboard.**

1.8 Proposed Solution Overview

> **How this section works:** a short preview of your solution (details come in Chapter 2). Mention the main components and how they connect. About half to one page.

The proposed system, **SmartFlow**, has four parts:

1. **Data layer:** historical PeMS readings are loaded into a Parquet data lake; live readings are published to an Apache Kafka topic; weather and holiday data are joined by time.
2. **Processing layer:** Spark SQL cleans the data and builds features such as lagged speeds, rolling averages, hour of day, day of week, rain and holiday flags.
3. **Learning layer:** Spark MLlib trains five models – Linear Regression, Decision Tree, Random Forest, Gradient-Boosted Trees and K-Means clustering – and the best model is selected using RMSE and MAE.
4. **Serving layer:** Spark Structured Streaming scores each new 5-minute reading, raises alerts for predicted heavy congestion, and a Power BI dashboard shows live and historical KPIs.

1.9 Objectives of the Project

> **How this section works:** 4–6 objectives, each starting with a verb (To design…, To compare…). They must be measurable, because in Chapter 5 you will check whether each one was achieved.

1. To build a Spark-based pipeline that ingests, cleans and stores at least five months of freeway sensor data in Parquet format.
2. To engineer temporal, weather and calendar features and study their relationship with congestion through exploratory analysis.
3. To train and compare at least five Spark MLlib models and select the best one using RMSE, MAE, MAPE and R^2 for speed and F1-score for congestion level.
4. To predict congestion in near real time (within one 5-minute interval) using Spark Structured Streaming.
5. To present KPIs, trends and predictions in a Power BI dashboard for traffic-management decisions.

1.10 Scope of the Project

> **How this section works:** say what is included AND what is not included. This protects you in the viva when someone asks "why didn't you do X?". About half a page.

In scope: freeway detector data from one PeMS district, 15/30/60-minute speed prediction, three-level congestion classification, local Spark cluster (single machine, multiple cores), a simulated live stream built from held-out data, and a Power BI dashboard.

Out of scope: arterial (city) roads with traffic signals, route recommendation for individual drivers, control of traffic signals, deep graph neural networks such as DCRNN [8], and deployment on a paid cloud cluster. These are listed as future work.

1.11 Industrial Applications and Use Cases

> **How this section works:** connect the project to real users and to the UN Sustainable Development Goals (required by the course). Bulleted use cases are fine. About 1 page.

- **Traffic control rooms:** early warning of congestion so that officers can adjust signal plans, deploy staff or update message boards before queues form.
- **Public transport operators:** better bus arrival estimates by knowing which freeway segments will slow down.
- **Logistics companies:** planning delivery departures to avoid predicted peaks, reducing fuel and driver hours.
- **Emergency services:** choosing ambulance routes that avoid segments predicted to be heavily congested.
- **Urban planners:** using long-term trends from the data lake to decide where road or transit investment is needed.

The project supports **SDG 11 (Sustainable Cities and Communities)** by improving urban mobility, **SDG 9 (Industry, Innovation and Infrastructure)** through data-driven infrastructure management, and **SDG 13 (Climate Action)**, because smoother traffic reduces idling and therefore fuel use and emissions [18].

Chapter 2: Project Design and Methodology

2.1 System Overview

> **How this section works:** a short, non-technical description of the whole system from the user's point of view: inputs, main processing, outputs. Half a page. The architecture diagram comes in 2.2.

SmartFlow takes three inputs – historical and live freeway sensor readings, hourly weather and a holiday calendar – and produces three outputs: a predicted speed for each sensor 15, 30 and 60 minutes ahead, a congestion level (Free-flow, Moderate, Heavy) and a dashboard of KPIs. Historical data is processed in batch to train models, and live data is processed as a stream using the trained model. Both paths use the same Spark code for cleaning and feature engineering, so the features seen during training and during prediction are identical.

2.2 Proposed System Architecture

> **How this section works:** insert your architecture diagram, give it a numbered caption, and explain it in 1–2 paragraphs. The text must refer to the figure ("as shown in Figure 2.1").

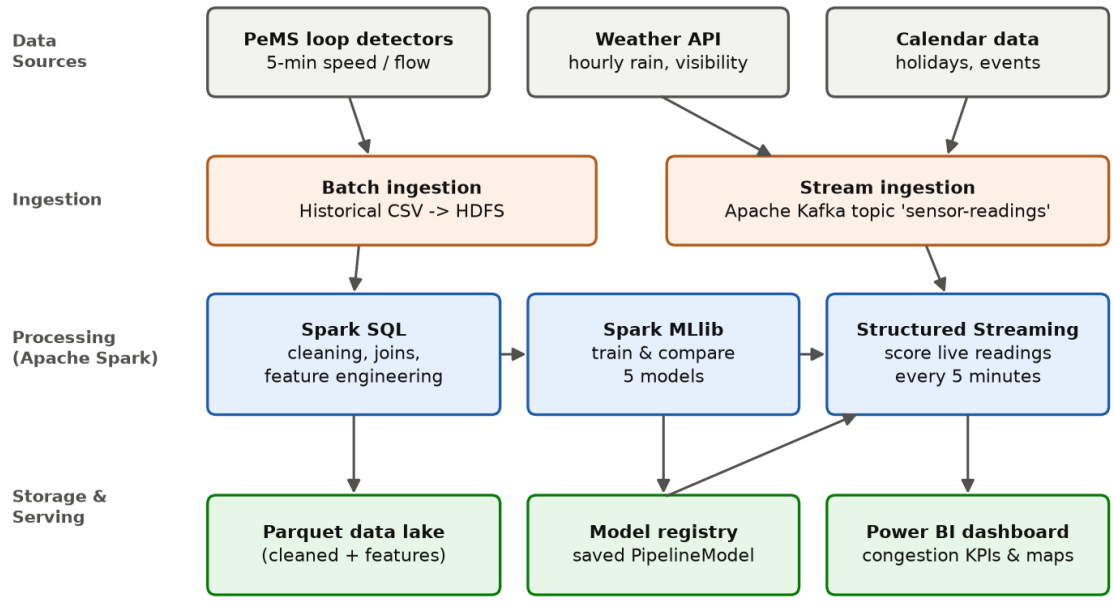


Figure 2.1 Proposed system architecture of SmartFlow

As shown in Figure 2.1, the architecture follows a layered design. Data sources at the top feed two ingestion paths: historical CSV files are copied to distributed storage for batch processing, while live readings are published to Kafka. In the processing layer, Spark SQL prepares the data, Spark MLlib trains and compares models, and Structured Streaming applies the selected

model to the live stream. Results are stored in a Parquet data lake and a model registry, and the Power BI dashboard reads the processed and predicted data.

2.3 Architecture Components and Their Functions

> **How this section works:** a table is the clearest way to list components. One row per box in the architecture diagram.

Table 2.1 Architecture components and their functions

Component	Function	Technology
Batch ingestion	Load historical sensor files into storage	Spark DataFrame reader, HDFS / local disk
Stream ingestion	Receive live 5-minute readings	Apache Kafka
Pre-processing	Handle missing values, outliers, time alignment, joins	Spark SQL
Feature store	Keep cleaned and engineered data	Parquet files
Model training	Train and compare 5 models	Spark MLlib Pipelines
Model registry	Save the selected PipelineModel	MLlib model save / load
Streaming scorer	Predict for each new reading, raise alerts	Spark Structured Streaming
Dashboard	KPIs, maps, trends, prediction accuracy	Power BI Desktop

2.4 Proposed Methodology

> **How this section works:** explain step by step HOW you will do the work, and WHY each step is needed. This is different from 2.5 (the diagram). Around 1–2 pages.

The methodology follows the standard data-science life cycle adapted for Big Data:

1. **Data collection:** download five months of 5-minute station data for one PeMS district, hourly weather for the same area and a list of public holidays.
2. **Pre-processing:** remove sensors with more than 20% missing readings, fill short gaps by linear interpolation, cap impossible values (for example speed above 100 mph) and align all sources to 5-minute timestamps.
3. **Exploratory data analysis:** study daily and weekly patterns, peak hours, the effect of rain and holidays, and correlations between flow, occupancy and speed.
4. **Feature engineering:** create lag features (speed 5, 10, 15 and 30 minutes earlier), rolling means over the last hour, hour of day, day of week, weekend and holiday flags, and rainfall.
5. **Model development:** split data by time (January–March for training, April for validation, May for testing) to avoid using the future to predict the past, then train the models listed in Section 2.6.

6. **Evaluation and selection:** compare models using the metrics in Section 2.11 and select the model with the lowest RMSE on validation data.
7. **Deployment:** replay the held-out month as a live Kafka stream, score it with Structured Streaming and show the results on the dashboard.

2.5 Methodology Workflow Use-case/DFD Flow Diagram

How this section works: show the methodology as a flow diagram and the users as a use-case diagram. Each figure needs a caption and a short explanation.

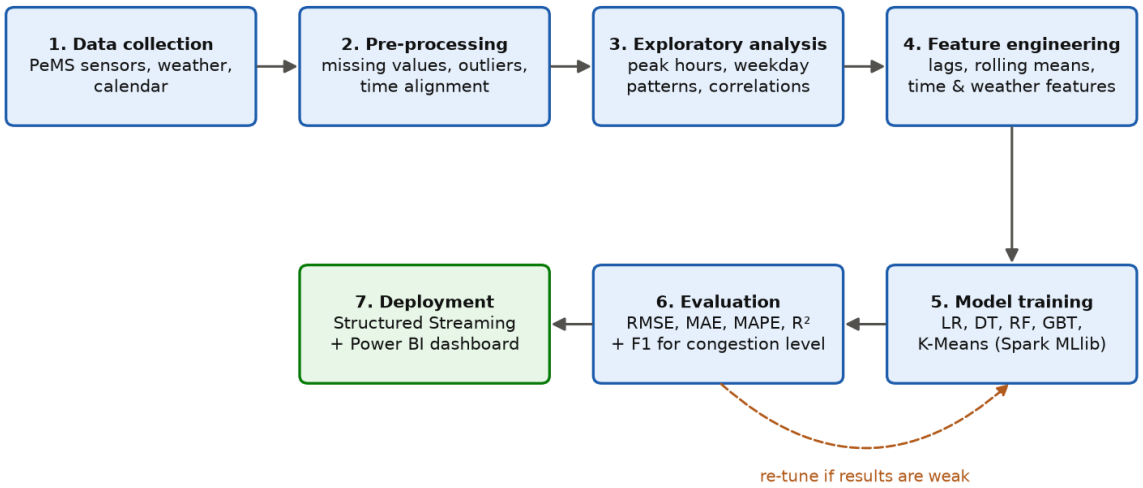


Figure 2.2 Methodology workflow

Figure 2.2 shows the seven stages of the workflow. The dashed arrow indicates that model training and evaluation are repeated with different settings until the results are acceptable.

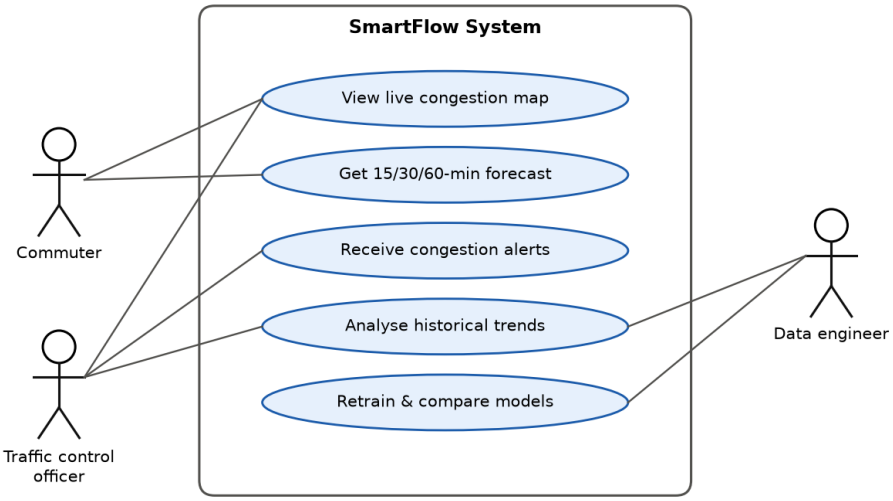


Figure 2.3 Use-case diagram

Figure 2.3 shows three actors. Commuters view the live map and forecasts; traffic control officers additionally receive alerts and analyse historical trends; the data engineer retrains and compares models when new data becomes available.

2.6 Algorithms and Techniques Used

> **How this section works:** for each algorithm: one-paragraph explanation in simple words + why you chose it. A summary table at the end helps. The course asks for at least 4–5 models implemented with PySpark.

Linear Regression predicts speed as a weighted sum of the input features. It is fast and easy to interpret and serves as the baseline.

Decision Tree Regressor splits the data using if-else rules on features (for example "hour between 7 and 9 and rain > 0"). It captures non-linear effects but can overfit.

Random Forest Regressor [10] trains many decision trees on random samples of rows and features and averages their predictions, which reduces overfitting.

Gradient-Boosted Trees (GBT) Regressor [11] builds trees one after another, where each new tree corrects the errors of the previous ones. It is often the most accurate model on tabular data.

K-Means clustering [16] groups sensors with similar daily speed profiles (for example "always congested in the morning"). The cluster label is used as an extra feature and for analysis on the dashboard.

Table 2.2 Algorithms used and reason for selection

Algorithm	Type	Reason for selection
Linear Regression	Regression	Baseline; fast and interpretable
Decision Tree	Regression	Non-linear rules; explainable
Random Forest	Ensemble regression	Robust to noise; less overfitting
Gradient-Boosted Trees	Ensemble regression	High accuracy on tabular features
K-Means	Clustering (unsupervised)	Discover groups of sensors with similar patterns

All five are available in Spark MLlib [2], so they are trained in a distributed way inside a Spark ML Pipeline that also contains the feature-assembly and scaling steps.

2.7 Dataset Description

> **How this section works:** give the exact source (with link/citation), size, time period, columns and license. A table is expected. Mention why the dataset is relevant and that it is faculty-approved.

The main dataset is freeway detector data from the **Caltrans Performance Measurement System (PeMS)** [13], [15], a public system maintained by the California Department of Transportation. Each record contains the timestamp, station identifier, total flow, average occupancy and average speed for a 5-minute interval. The widely used PEMS-BAY subset [8] covers 325 sensors in the San Francisco Bay Area from January to May 2017.

Table 2.3 Dataset description

Attribute	Details
Source	Caltrans PeMS (public), PEMS-BAY benchmark subset [8], [13]
Time period	1 January – 31 May 2017
Granularity	5-minute intervals (12 readings per hour per sensor)
Sensors	325 freeway detectors
Approximate size	≈ 17 million sensor-interval records (325 sensors × ≈52,000 intervals)
Main columns	timestamp, station_id, flow, occupancy, speed, lane count, latitude, longitude
External data	Hourly weather (rain, visibility); California public holidays
Target variables	Speed after 15/30/60 min (regression); congestion level (classification)

2.8 Data Collection and Data Preprocessing

> **How this section works:** describe exactly how the data is obtained and every cleaning step, in order, with the reason for each. You may include a small code-free example ("a speed of 0 during free-flow hours is treated as a sensor error").

Collection. Station 5-minute files are downloaded from the PeMS website [15] as compressed text files, one per day, and loaded with Spark's CSV reader using an explicit schema. Weather is downloaded as hourly CSV and holiday dates are entered as a small reference table.

Pre-processing steps (all implemented with Spark DataFrame operations):

- **Schema enforcement:** read every column with a fixed data type so that malformed rows are detected early.
- **Missing values:** drop stations with more than 20% missing intervals; fill gaps of up to three intervals (15 minutes) by linear interpolation using window functions; drop longer gaps.
- **Outliers:** treat speed above 100 mph or below 0, and flow above the physical lane capacity, as sensor errors.
- **Time alignment:** round all timestamps to the 5-minute grid and join weather by hour and holidays by date.

- **Feature scaling:** apply StandardScaler inside the MLlib pipeline for Linear Regression and K-Means.
- **Storage:** write the cleaned table in Parquet format partitioned by date for fast reading.

2.9 Tools and Technologies Used

> **How this section works:** list software with versions and the purpose of each. Examiners like a table. Include hardware if relevant.

Table 2.4 Tools and technologies

Category	Tool / Version	Purpose
Distributed processing	Apache Spark 3.5 (PySpark)	Batch processing, SQL, ML, streaming
Machine learning	Spark MLlib	Regression, ensembles, clustering, pipelines
Streaming	Spark Structured Streaming, Apache Kafka	Real-time ingestion and scoring
Storage	Parquet, HDFS (optional)	Columnar, compressed data lake
Visualisation / BI	Power BI Desktop, Matplotlib	Dashboard and report charts
Language	Python 3.12	Pipeline and analysis code
Runtime	Java 17	Required by Spark
Version control	Git, GitHub	Code management and collaboration

2.10 Experimental Setup and Configuration

> **How this section works:** describe the machine, Spark settings and the data split you WILL use. For Milestone 1 this is a plan; you confirm the final values in Chapter 4.

Table 2.5 Experimental configuration

Item	Configuration
Hardware	Laptop, 8-core CPU, 16 GB RAM, SSD
Spark mode	local[*] (all cores); same code can run on a cluster
Driver memory	6 GB
Train / validation / test split	By time: Jan–Mar train, Apr validation, May test
Hyper-parameter tuning	TrainValidationSplit on the validation month
Streaming trigger	Every 5 minutes (micro-batch), replaying the test month
Random seed	42 (for reproducibility)

2.11 Evaluation Metrics

> **How this section works:** define each metric with its formula and say what "good" looks like. Explain why you chose these metrics for YOUR problem. About 1 page.

Speed prediction is a regression task, so the following metrics are used, where y_i is the actual speed, $\hat{y}_i$ the predicted speed and n the number of test records:

Table 2.6 Evaluation metrics

Metric	Formula	Meaning
MAE	$(1/n)\ \Sigma\ \lvert y_i - \hat{y}_i \rvert$	Average error in mph; easy to explain
RMSE	$\sqrt{[(1/n)\ \Sigma\ (y_i - \hat{y}_i)^2]}$	Penalises large errors more (important for sudden jams)
MAPE	$(100/n)\ \Sigma\ \lvert y_i - \hat{y}_i \rvert / y_i$	Error as a percentage of actual speed
R^2	$1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$	Share of variation explained by the model (1 = perfect)
F1-score	$2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$	For the Heavy congestion class, which is rare

RMSE is the primary metric for model selection because missing a sudden drop in speed is more harmful than several small errors. For the congestion-level classification, accuracy alone is misleading because Heavy congestion is much rarer than Free-flow, so the F1-score of the Heavy class is reported. Training time is also recorded to discuss scalability.

References

> **How this section works:** use one citation style consistently (IEEE is common in engineering). Every [n] in the text must appear here, and every entry here must be cited in the text. Only list sources you have actually read.

[1] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," Communications of the ACM, vol. 59, no. 11, pp. 56–65, 2016.

[2] X. Meng et al., "MLlib: Machine learning in Apache Spark," Journal of Machine Learning Research, vol. 17, no. 34, pp. 1–7, 2016.

[3] M. Armbrust et al., "Structured Streaming: A declarative API for real-time applications in Apache Spark," in Proc. ACM SIGMOD, 2018, pp. 601–613.

[4] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," Communications of the ACM, vol. 51, no. 1, pp. 107–113, 2008.

[5] B. M. Williams and L. A. Hoel, "Modeling and forecasting vehicular traffic flow as a seasonal ARIMA process: Theoretical basis and empirical results," Journal of Transportation Engineering, vol. 129, no. 6, pp. 664–672, 2003.

[6] Y. Lv, Y. Duan, W. Kang, Z. Li, and F.-Y. Wang, "Traffic flow prediction with big data: A deep learning approach," IEEE Transactions on Intelligent Transportation Systems, vol. 16, no. 2, pp. 865–873, 2015.

[7] X. Ma, Z. Tao, Y. Wang, H. Yu, and Y. Wang, "Long short-term memory neural network for traffic speed prediction using remote microwave sensor data," Transportation Research Part C, vol. 54, pp. 187–197, 2015.

[8] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in Proc. International Conference on Learning Representations (ICLR), 2018.

[9] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in Proc. IJCAI, 2018, pp. 3634–3640.

[10] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[11] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[12] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in Proc. NetDB Workshop, 2011.

[13] C. Chen, K. Petty, A. Skabardonis, P. Varaiya, and Z. Jia, "Freeway performance measurement system: Mining loop detector data," Transportation Research Record, no. 1748, pp. 96–102, 2001.

[14] Apache Software Foundation, "Machine Learning Library (MLlib) Guide," [Online]. Available: https://spark.apache.org/docs/latest/ml-guide.html

[15] California Department of Transportation, “Caltrans Performance Measurement System (PeMS),” [Online]. Available: https://pems.dot.ca.gov/

[16] S. P. Lloyd, “Least squares quantization in PCM,” IEEE Transactions on Information Theory, vol. 28, no. 2, pp. 129–137, 1982.

[17] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, “The Hadoop distributed file system,” in Proc. IEEE MSST, 2010.

[18] United Nations, “Transforming our world: The 2030 Agenda for Sustainable Development,” A/RES/70/1, 2015.